

Deep learning-based intrusion detection for IoT - an approach assisted by formal modeling and mutation testing

Yara Kaddoura , Khaled El-Fakih*, Imran Zualkernan, Alifiya Bhanpurawala

Department of Computer Science and Engineering, American University of Sharjah, Sharjah, 26666, UAE

ARTICLE INFO

Keywords:

Internet of Things
Intrusion detection
Formal modeling
Mutation testing
Timed state machines
Deep learning
Long short-term memory model
Transformer model

ABSTRACT

Deep learning and machine learning techniques have been increasingly adopted in intrusion detection systems (IDS) to detect and identify IoT security threats. However, traditional methods for training such systems require large amounts of data collection, making them inefficient and difficult to tailor to specific devices. This paper proposes a novel formal approach that leverages formal modeling and mutation testing to intelligently generate network data required for training deep-learning-based IDS systems. The approach is demonstrated in the context of four commonly used sensors; a temperature sensor, a motion sensor, an ultrasonic motion sensor, and an RFID card reader. Each device is modeled as a timed state machine that is systematically mutated to model various attack types including man-in-the-middle, replay, data falsification, and battery draining attacks. Mutation testing is then used to generate a concise yet representative training dataset capable of distinguishing the mutated and the original machines. The corresponding network data is recorded, parsed, and used to train two deep learning models - a long short-term memory (LSTM) and a transformer-based model- to classify these different attacks. To evaluate the effectiveness of our approach, we compare it against a proposed baseline approach that externally simulates attacks and passively collects network data. Experiments show that our formal approach significantly outperforms the baseline across all evaluation metrics, achieving an average F1-score improvement of 51.8% and 60.3% for the LSTM and transformer models. In addition, our approach reduces the number of test cases derived on average by 95.5%, demonstrating its efficiency in generating high-quality device-specific IDS training data.

1. Introduction

The Internet of Things (IoT) consists of interconnected, sensor-based physical devices that exchange data over the Internet or other networks [1]. The number of IoT devices has been growing exponentially, with an estimated total of 30.9 billion devices expected to be connected to the internet by 2025 [2]. These devices are widely deployed across various domains including smart homes, healthcare, industrial automation, transportation and smart cities [3]. However, this rapid expansion has also introduced a myriad of security challenges. IoT devices are often resource-constrained, limiting their ability to implement robust security measures, which makes them prime targets for malicious attackers and poses substantial risks to their data privacy, integrity, and availability. Many of these devices also suffer from weak authentication mechanisms, allowing unauthorized attackers to intercept network traffic and compromise data integrity.

* Corresponding author.

E-mail addresses: ykaddoura@aus.edu (Y. Kaddoura), kelfakih@aus.edu (K. El-Fakih), izualkernan@aus.edu (I. Zualkernan), g00081958@aus.edu (A. Bhanpurawala).

<https://doi.org/10.1016/j.iot.2026.101958>

Received 19 April 2025; Received in revised form 16 February 2026; Accepted 24 April 2026

Available online 26 April 2026

2542-6605/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

As a result, IoT systems are vulnerable to a wide range of attacks including physical attacks (such as node tampering or injection, which injects malicious code into the node functionality to gain control over the node [4]), power-related attacks (such as sleep deprivation, where the attacker forces the node to consume excessive power until it shuts down), and network-based attacks (such as Radio Frequency Identification (RFID) spoofing and cloning, which inject false information that the system accepts as legitimate, and sinkhole attacks, where a compromised node is used to disrupt network traffic by dropping or modify packets) [5]. Additionally, man-in-the-middle (MITM) attacks occur when an attacker intercepts and manipulates communications between IoT nodes, compromising data integrity and confidentiality. Similarly, Sybil attacks occur when a malicious node impersonates multiple legitimate ones to influence network behavior [5]. Other threats include denial-of-service attacks, which render any offered services unavailable by flooding the network with traffic, and malware-based threats such as viruses, worms, and spyware, which compromise devices and their provided services. IoT devices are also vulnerable to side-channel attacks, which extract encryption keys to decrypt sensitive information transmitted by the device. Most of these attacks are active attacks with a high damage level that pose serious threats to the integrity, confidentiality, and availability of IoT systems, making effective security measures imperative [5].

Several strategies have been proposed to enhance IoT system security and mitigate these risks. Security best practices, such as end-to-end encryption and key management techniques, help prevent attacks such as node injection and eavesdropping [4]. However, countering attacks such as side-channel attacks requires modern hardware with secure boot processes and up-to-date software. In addition, intrusion detection systems (IDS), key and trust management, and rank verification play a crucial role in maintaining IoT network security.

Deep learning (DL) has emerged as a powerful tool for detecting and classifying cyberattacks in IDS and has been widely explored in research. However, its effectiveness is highly dependent on the quality and quantity of training data. IDS models require datasets that are not only representative of real-world attacks but also diverse enough to allow generalization and prevent overfitting. The size and availability of high-quality datasets pose additional challenges, as collecting and labeling large-scale, attack-specific IoT data is labor-intensive and difficult to maintain with evolving threats.

In this paper, we propose and evaluate both a baseline and a novel formal approach for IoT network data collection to train deep learning models for classifying security threats in IDS. Unlike existing methods that passively collect network data by externally interacting with a system – simulating attacker behavior and recording responses for training – the formal approach we propose actively and intelligently generates targeted datasets using a formal model of the IoT device under observation. Our formal approach introduces a novel technique for generating IDS training data that is both device-aware and highly efficient. Rather than randomly collecting large amounts of network traffic, we use formal modeling and mutation testing to systematically generate a smaller representative dataset of attack data specific to the IoT device's design and behavior. Formal modeling provides a mathematically rigorous framework for representing complex systems, allowing for precise verification of system properties. Model-based mutation testing then introduces controlled faults (mutants) that simulate different attack scenarios and allows for the derivation of targeted test cases to detect them. By leveraging this intelligent data generation process, our approach enables a more accurate threat classification and faster detection while significantly reducing the amount of data required for effective IDS training. We apply this methodology to IoT devices with simple designs and minimal overhead, such as temperature sensors, RFID card readers, and motion sensors. Our goal is to shift IDS data collection from a tedious, brute-force process to a structured, model-driven approach that enhances both efficiency and effectiveness in detecting security threats. In addition, to ensure a fair and meaningful comparison, we designed our own baseline method. Proper evaluation requires that both the baseline and our formal approach operate within the same paradigm, meaning that the baseline must collect data using conventional methods without device modeling, while our approach must leverage the model to generate intelligent datasets. However, as will be detailed in Sections 2 and 6, existing datasets cannot be applied to our work, as they do not meet the necessary criteria for a meaningful evaluation; for instance, they either lack the key training features used by our approach, do not cover the attack types considered in this work, or involve different devices and IoT environments. As a result, we developed a baseline method that aligns with conventional data collection strategies, allowing for a controlled and objective comparison.

We model the considered IoT devices using the timed finite-state machine (FSM) and extended FSM models. FSMs are used to model reactive systems whose behavior can be represented using a finite number of states and transitions between them. For modeling complex systems, the extended FSM model can be used, as it extends the FSM model with variables, predicates, and update statements. An extended FSM can have variables, and a transition can be guarded by a predicate (guard) that must hold for the transition to be executed, and upon its execution, new values can be assigned to the variables based on the transition update statements. State machines are used in various application areas and act as the underlying models for a number of industrial strength specification techniques, such as Statecharts [6], SDL [7] and UML [8]. For modeling the time behavior of IoT devices, we use the timed version [9–11] of the state machine models. A timed machine has a single clock variable that is reset to 0 upon the execution of any transition, and the machine can also have timeout transitions at states such that if no input is applied at the current state before the transition timeout expires, then the timeout transition is executed, and the machine resets the value of the clock to 0 while moving to the designated state. The reader may refer to Merayo et al. [9], Hierons et al. [10], Bresolin et al. [11], Petrenko et al. [12], Bourhfir et al. [13], Kalaji et al. [14] for formal definitions and examples of such machines. We show how such timed state machine models can be used for modeling both the behavior of the considered IoT devices as well as the behavior-related attacks. The attacks are modeled by introducing slight variations (mutants) to the original behavior to simulate a wide range of potential anomalous behaviors. Afterwards, we use mutation testing to derive tests [11,12,15–18] that can distinguish the original behaviors from the malicious ones; i.e., detect the malicious behaviors. The derived tests, which are concise (very few in number), are injected into the system and used to train the considered machine learning models. Experiments show that models trained using these tests have classification superiority over those derived using a baseline approach.

The contributions of this work are as follows.

- We formally model the temperature sensor, motion sensor, ultrasonic motion sensor, and RFID card reader as timed finite state machines (TFSMs) and extended TFSMs. Additionally, we target a group of common IoT attacks including sleep deprivation, battery draining, MITM, and replay attacks, and intelligently model these attacks as special mutants of these IoT devices' TFSMs.
- We use the TFSM models of the modeled devices and their attack mutants to derive test cases capable of detecting these attacks. We also develop a baseline method that externally simulates the considered attacks, enabling a controlled comparison. The tests derived from both methods are used to generate network data for training the considered deep-learning models.
- We design and implement a testbed/environment to execute our proposed models and collect the necessary network data. We use the tests obtained from the formal and the baseline approaches to collect the related network data. Feature extraction and engineering of the collected network data are then used to train the deep-learning models.
- We implement two deep learning models, a long short-term memory (LSTM) and a transformer model to perform multi-class classification on datasets generated by both approaches.
- We experimentally demonstrate the effectiveness of both the baseline and formal approaches and cite the superiority of the formal approach against the baseline method.

This paper is organized as follows. [Section 2](#) includes previous related work and [Section 3](#) introduces related preliminaries. [Sections 4 and 5](#) include the modeling of IoT devices and their attacks. Classification of the attacks using formal and baseline methods with related LSTM and transformer models is given in [Section 6](#). [Section 7](#) includes the experimental evaluation, and [Section 8](#) concludes the paper.

2. Related work

2.1. IoT datasets

Several datasets have been introduced to evaluate IDS systems containing IoT devices using Machine Learning (ML) models. The IoT Network Intrusion Dataset, collected by Kang et al. [19], contains packet files on different types of IoT-based network attacks including denial of service (DOS), botnet, scanning and MITM attacks. The tool nmap was used to simulate all the attacks except for the Mirai botnet category, which was manually generated by manipulating packets to appear as if they originated from an IoT device. Ullah et al. [20] further built on the dataset in Kang et al. [19] to produce a new dataset, IoTID20. Another dataset, the TON_IoT dataset [21], includes data collected from multiple sources such as IoT and IIoT sensors, operating systems and network traffic covering nine attack categories. Agent systems were used as a testbed to create realistic normal and attack scenarios. Similarly, IoT_23 [22] is a recent dataset containing network traffic data from IoT devices. It includes a benign category and eight attack types, such as Okiru botnet, distributed denial of service (DDoS) and Mirai botnet. The collected data is in the form of network captures of IoT network traffic from three smart devices. Some datasets specifically focus on IoT networks that utilize MQTT as a communication protocol. For example, the MQTT-IoT-IDS2020 dataset, presented by Hindy et al. [23], simulated an MQTT-based IoT system and contained data from four attacks: aggressive scan, UDP scan, sparta SSH brute-force and MQTT brute-force. This dataset primarily focuses on the types of data transferred between the sensors rather than the actual messages, which were randomly generated. Koroniotis et al. [24] proposed BoTIoT, a dataset that incorporated legitimate and simulated IoT network traffic with different types of attacks including DDoS, DoS, OS and service scanning, keylogging and data exfiltration attacks. They used the Ostinato traffic generator tool to generate network traffic and the Node-RED middleware to simulate the presence of IoT devices in their network using MQTT messages. Additionally, Vaccari et al. [25] developed a dataset focused on MQTT-based IoT networks, simulating four types of attacks: DoS, SlowITe, malformed data and brute force authentication. Their network testbed included eight sensors, each programmed to produce periodic or random messages.

Although all of the aforementioned datasets contain network security attack data, most do not use MQTT as a communication protocol [19,21,24], while our dataset specifically focuses on IoT devices communicating via MQTT. Furthermore, data collection and traffic generation methods in all these datasets [19,21–25] are often unspecified, randomized, or conducted ad hoc. In contrast, our approach employs a structured methodology, generating concise datasets through finite-state modeling of the behavior of the IoT device under attack. This ensures a more systematic, reproducible and controlled dataset for evaluating attack detection and classification. In addition, Jamal et al. [26] show that deep and hybrid deep models (e.g. CNN-DBN and CNN-LSTM) achieve the best performance for attack detection when appropriate re-sampling strategies are applied, highlighting the importance of data preparation rather than relying only on the choice of detection models used. Furthermore, while the datasets in Kang et al. [19], Alsaedi et al. [21], Garcia et al. [22], Hindy et al. [23], Koroniotis et al. [24], Vaccari et al. [25] provide valuable insights into IoT security, they are not suitable as baselines for our study. As discussed in [Section 6](#), they either lack the key training features used by our approach, do not cover the attack types we classify, or involve different IoT device environments. These limitations make direct comparison infeasible, necessitating the development of our own baseline method.

2.2. Neural networks performance on IoT datasets

According to recent literature, some models like the Recurrent Neural Networks (RNN), have performed comparatively well with respect to the classification of time series data such as IoT network intrusion datasets. For example, Azumah et al. [27] trained

an LSTM model on the IoT Network Intrusion dataset and reported F1-score and accuracy results of 83% and 98%, respectively. Similarly, Elsayed et al. [28] proposed a BiLSTM-CNN hybrid model trained on the same dataset and reported an F1-score of 98%. Islam et al. [29] implemented two types of RNN models on IoT network datasets: an LSTM and a stacked LSTM. The best results reported for both models were 99%. Alkahtani et al. [30] also proposed a CNN-LSTM hybrid model and reported an accuracy of 98% when trained on the IoTID20 dataset. Alsaedi et al. [21] trained multiple models on their dataset, TON_IoT, including an LSTM which achieved an F1-score of 80%. Ullah et al. [31] trained three types of RNN models: an LSTM, BiLSTM and GRU models on a number of datasets including IoT_23, BoT_IoT and MQTTset. The LSTM model scored an average F1-score of 97%, 99% and 99% for the three datasets respectively for multiclass classification. Similarly, the BiLSTM scored an average F1-score of 97%, 99% and 99% for the three datasets while the GRU scored 98%, 99% and 98% respectively. Saurabh et al. [32] proposed a stacked LSTM for multiclass classification and trained it on the BoT_IoT dataset, reporting a weighted average F1-score of 100%. In addition, Jamal et al. [33] proposed a hybrid CNN-DBN intrusion detection model for industrial networks and report strong multi-class attack detection results on the MSCAD dataset, further demonstrating that machine-learning and deep-learning models can be highly effective when applied to IDS datasets.

2.3. Use of mutation testing in machine learning

Most of the existing research on the use of formal methods and mutation testing to train machine/DL models is centered on robust training or adversarial training. Adversarial training is a technique used to enhance the robustness of ML and DL models against adversarial attacks, which encompass small perturbations made to the input data that cause the model to produce incorrect classifications. For example, Dalvi et al. [34] developed a framework to increase the robustness of a Naive Bayes classifier for detecting spam emails to prevent attackers from actively trying to make the trained classifier produce false negatives. They achieve that by injecting modifications into the features of the training data through an adversary to produce an adversary-aware classifier that performs significantly better. Similarly, Moosavi-Dezfooli et al. [35] proposed an algorithm that accurately computes slight adversarial perturbations in the input data to fool deep network classifiers. Their algorithm is based on an iterative linearization of the neural network to generate small perturbations that are enough to change the classification label. They highlight that this can be reliably used to fine-tune the classifiers to be more robust. Calzavara et al. [36] argue that the original formulation of adversarial training cannot be applied directly to all ML models, including decision tree ensembles. Therefore, they presented a method to increase the robustness of gradient-boosted decision trees against adversarial attacks by generalizing the original approach to adversarial training. Finally, Naeem et al. [37] proposed a novel approach to improve the efficiency of software mutation testing by using predictive analysis techniques in DL models. The proposed framework predicts the effectiveness of a mutation by analyzing specific features extracted from the mutants, such as reachability, infection, and propagation. A DL model is then trained to predict effective mutations, leading to a more efficient testing process.

3. Background

3.1. Timed state machines

In this subsection, we introduce the timed state machine model which we use to precisely represent the behavior of IoT devices and their attacks. A Finite State Machine (FSM) model of a system is specified as a finite set of states, inputs, outputs, and transitions between states each labeled by an input/output interaction. Formally, an FSM is a 5-tuple (S, I, O, h, s_1) where I and O are input and output alphabets, S is a finite non-empty set of states with the designated initial state s_1 , and $h \subseteq (S \times I \times O \times S)$ is the transition relation. A transition in h , given in the form (s, i, o, s') indicates that if the machine is in the current state s , upon receiving the input i , it produces the output o while moving to state s' .

An FSM with timeouts [11,38], a timed FSM (TFSM) for short, is an FSM annotated with a clock that is reset to 0 upon the execution of any transition and can have input timeout transitions. When an input timeout expires at a state, the TFSM can spontaneously move to the destination state of the timeout transition while resetting the clock to 0. Thus, a TFSM is a 6-tuple $Q = (I, S, O, h, \Delta, s_1)$ where $\Delta: S \rightarrow S \times (N \cup \infty)$ is the timeout function, where N is the set of positive integers: for each state, this function specifies the maximum time to wait for an input. Given state s of Q such that $\Delta(s) = (s', t_{out})$, if no input is applied before t_{out} expires, then at t_{out} , Q moves to state s' and the clock is set to 0. If $s = s'$ then the clock is set to zero when the timeout has expired. A transition $(s, i, o, s') \in h$ means that Q being at state s accepts an input i applied at time $t < t_{out}$, measured from the moment when the clock was reset at state s , then Q produces o and the clock then set to 0.

Given a TFSM Q , a timed input is a pair (i, t) where i is in I and t is a real number; a timed input (i, t) means that input i is applied to Q at the time instant t where t is a local time at the state. A timed input of a timeout transition is written as (ϵ, t_{out}) and its output is ϵ , where ϵ is the empty string. A sequence of timed inputs $\alpha = (i_1, t_1)(\epsilon, t_{out}) \dots (i_n, t_n)$ is a *timed input sequence*. A timed sequence $\alpha/\gamma = (i_1, t_1)/o_1(\epsilon, t_{out})/\epsilon \dots (i_n, t_n)/o_n$ of consecutive pairs of timed inputs and outputs starting at the state s is a (timed) trace of Q at state s . A trace of Q starts from the initial state s_0 . The input sequence of the trace is called a *test case* of TFSM M and the number of inputs in I of this sequence is the *length* of the test case, and the sum of the timeouts of a test case is the *total time* of the test case. A test suite is a finite set of test cases. For simplicity, in the figures, we use $s - t_{out} \rightarrow s'$ to denote the timeout transition from s which leads at t_{out} to state s' .

For example, consider the TFSM of the temperature sensor in Fig. 2. At the initial state S_1 , the clock starts from 0, then at time $t_{out} = 2$ the machine moves to state S_2 while resetting the clock to 0. Then at S_2 if i_1 is not provided at time $t < t_{out} = 2$, then at

$t = 2$, t_{out} is executed and the machine moves back to S_1 while resetting the clock to 0. However, if at S_2 the input i_1 is provided at $t < t_{out} = 2$, then the machine executes transition t_4 where it produces o_2 while moving to state S_3 and resetting the clock to 0. The traces that correspond to these two scenarios are $(\epsilon, t_{out} = 2)(\epsilon, t_{out} = 2)$, and $(\epsilon, t_{out} = 2)(i_1, t < 2, o_1)$, respectively.

3.2. Extended TFMSs

An Extended TFMS builds upon the traditional TFMS by incorporating variables, conditions, and actions, allowing for more expressive modeling of complex time-dependent behaviors. We use the extended TFMS model to model the behavior of the ultrasonic motion sensor and the RFID reader. For the ultrasonic motion sensor (shown in Fig. 4), as specified by transition t_{21} , the input i_{11} carries the input parameter variable "duration", which is used to update the variable "distance" upon the execution of the transition where the machine outputs o_{11} while moving to state S_9 and resets the clock to 0. Thus, a machine can have parameterized inputs and variables, and the variables can be updated upon the execution of transitions. In addition, a transition can have a guard, such that if the input of the transition is received, the transition can then be executed only if the guard is satisfied. For example, the RFID reader model given in Fig. 5 has the variable "counter" and this variable is used as the guard [$counter < 10$] of transition t_{22} . Thus, at state S_8 , if the input i_{11} is received then t_{22} can be executed only if the guard [$counter < 10$] holds. If executed, then the update statements of the transition are run and afterwards the output o_{13} is produced and the machine moves to S_9 while resetting the clock to 0.

3.3. Neural networks

Artificial Neural Networks (NN) are computing systems originally inspired by how the human brain works, designed to imitate the way biological neurons transmit signals to one another [39]. They are a subset of ML and the center of DL algorithms. Recurrent NN (RNN) are special NNs that preserve sequential information to be able to make future predictions [40]. These networks are used in a large number of applications, such as natural language processing and in the analysis of time-series data, where past information is crucial to the prediction of future information.

In this paper, we utilize two types of DL models. The first one is an LSTM model [41], since LSTMs are better at handling long-term dependencies, making it easier to remember past information when compared to standard RNNs. An LSTM consists of an input gate, a memory cell, and an output gate [27] and is modeled by building a linear stack of layers. We also consider a transformer model, a type of DL model that leverages self-attention to weigh the importance of different time steps which allows it to effectively capture temporal dynamics [42]. This makes such models well-suited for time series classification tasks, where the ability to model complex temporal dependencies and relationships is crucial. In addition, the parallelizable architecture of transformers allows for more efficient training.

4. Modeling IoT edge services

In this section, we describe the functionality of a temperature sensor, a motion sensor, an ultrasonic motion sensor, and an RFID card reader and model their behavior using TFMSs. We outline how each device transitions between states in response to inputs, capturing their expected behavior in a structured and formalized manner.

System Architecture: In our system, the sensor is connected to an ESP32 microcontroller [43] that uses WiFi to connect to the home network router and to the internet. Once the sensor detects an event (e.g. the motion sensor detects motion), the microcontroller uses the MQTT protocol to send a message to the application server and to any other applications running on a consumer's machine or mobile phone. The architecture is shown in Fig. 1. From a communication perspective, the microcontroller must first connect to the WiFi network and then connect to an MQTT broker using TCP. In our case, we use the mosquitto MQTT broker. Once connected to the WiFi and MQTT broker, the microcontroller will continuously scan for signals from the sensors. After a signal is detected, the microcontroller will debounce by sleeping for a few milliseconds. This ensures that one event does not generate multiple duplicate messages. For each event, an MQTT message is then published.

Temperature sensor TFMS: Here, we model the temperature sensor as a TFMS and demonstrate how this model can be adapted to represent the behavior of the remaining sensors. The temperature sensor is a classic IoT device that continuously measures the temperature of the surrounding environment and converts the input data to an electrical signal. After the microcontroller connects to the WiFi network and the MQTT broker, it publishes the measured temperature from the sensor in an MQTT message once every 2 seconds. The TFMS that represents the sensor is shown in Fig. 2. The detailed descriptions of its inputs and outputs are provided in Tables 1 and 2, respectively. The inputs represent input messages received by the system, either from MQTT or as system connect checks. The output messages are printed to the console and published as MQTT messages. The microcontroller first waits for 2 seconds before it connects to the WiFi by moving from the initial state S_1 to S_2 using transition t_1 . Note that when a machine executes a transition and moves to the intended state the clock is reset to 0. Then if the microcontroller receives the input $i_2 = \text{ws}(1)$, signaling a successful connection to the WiFi, it outputs the message o_2 to the console and moves to the connected state S_3 ; however, if it receives the input $i_1 = \text{ws}(0)$, signaling a failed attempt at connecting to the WiFi, then it outputs the message o_1 while moving back to S_2 . In all cases, the machine returns back to state S_1 using transition t_2 , if the inputs i_1 and i_2 are not received within 2 seconds after reaching state S_2 . Then after the microcontroller has connected to the WiFi in state S_3 , it waits for 2 seconds before moving to S_4 . Timeouts were added to all states to ensure that the microcontroller does not get stuck in any state. In state S_4 , the microcontroller will attempt to connect to the MQTT broker. If it receives the input i_3 which signals a failed connection attempt, then it will return,

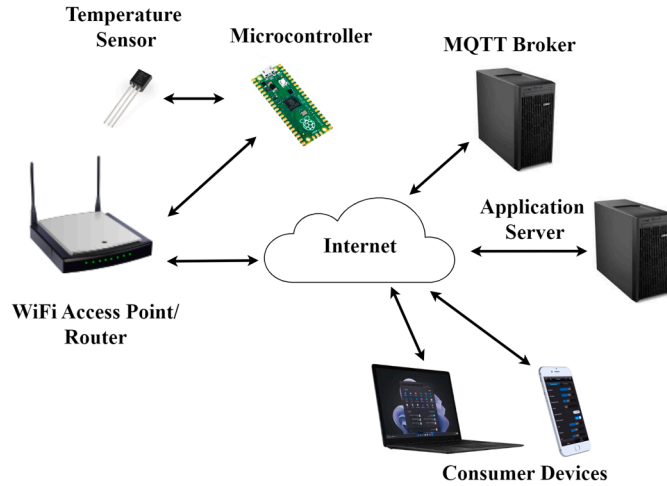


Fig. 1. System architecture.

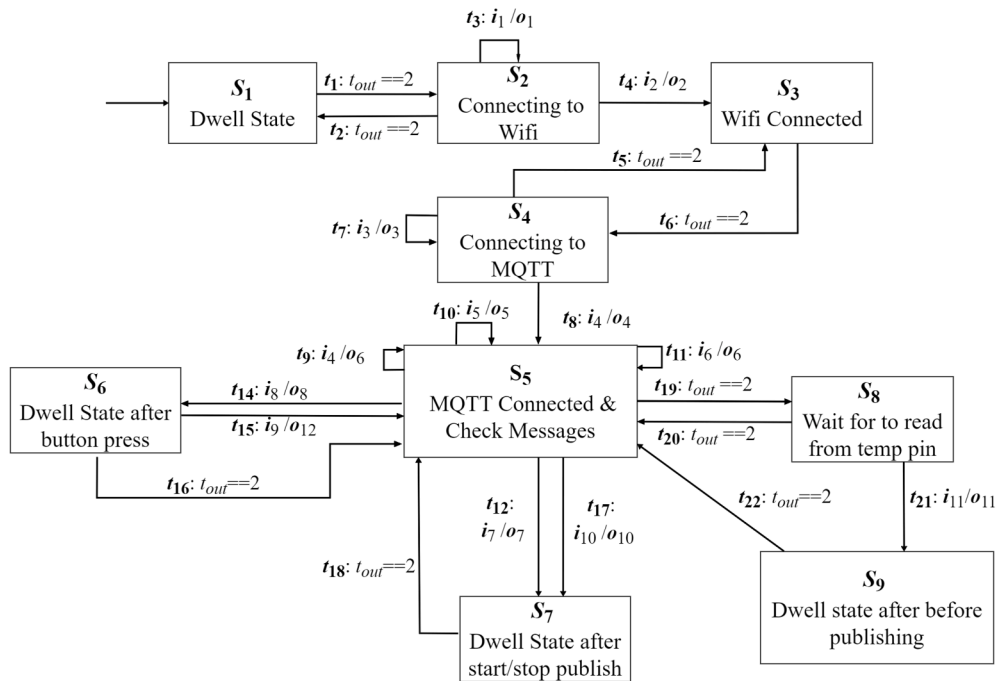


Fig. 2. Temperature sensor TFSM.

using t_7 , to state S_4 to reconnect. However, if it receives the input $i_4 = cc(1)$, signaling a successful connection to the MQTT broker, it then outputs $o_4 = \text{"Connected to MQTT"}$ and moves to state S_5 as specified by transition t_8 . Then at S_5 , once connected to the WiFi and MQTT broker, the microcontroller cycles between states S_5 , S_6 , S_7 , S_8 , and S_9 . Since it is single-threaded, the microcontroller must check for any incoming messages from the MQTT broker in S_5 while also periodically reading the temperature in states S_8 and S_9 . In state S_5 , there are many possible input events, such as input $i_5 = \text{"Healthy Check"}$, which inquires about the sensor status, and the response should be that the sensor is alive. There are also inputs that would lead to different dwell states, including the input event i_7 which instructs the microcontroller to stop publishing any messages for 2 seconds. After the input is received, the output message $o_7 = \text{"Publishing has stopped"}$ is published and the system enters a dwell state S_7 where the microcontroller sleeps for 2 seconds before returning to state S_5 to check for any other incoming messages. Another input event is $i_8 = \text{"Give me a WiFi Name"}$, which requests the name of the WiFi network the microcontroller is connected to. This event outputs $o_8 = \text{"Press any button"}$ and enters a dwell state S_6 . If the input i_9 is received before the 2 second timeout, then the microcontroller publishes the name of the WiFi network and returns to S_5 . Otherwise, if the timeout event occurs, then the system returns to S_5 without outputting the WiFi network name. However, if the system remains in state S_5 and receives no inputs within the 2 second timeout limit, then it moves to state S_8 to take the

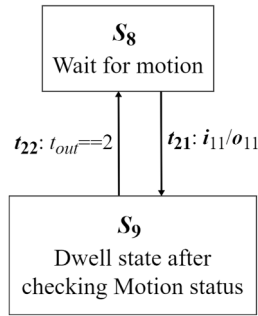


Fig. 3. Part of motion sensor TFSM. The other part is similar to the temperature sensor TFSM given in Fig. 2.

Table 1

Inputs and outputs common to all TFSMs (sensors).

Input	Description	Output	Description
i_1	ws(0)	o_1	"Connecting to wifi"
i_2	ws(1)	o_2	"Connected to wifi"
i_3	cc(0)	o_3	"Connecting to MQTT"
i_4	cc(1)	o_4	"Connected to MQTT"
i_5	"Healthy Check"	o_5	"Sensor is alive"
i_6	"Unexpected Request"	o_6	"Error.Invalid Message"
i_7	"Stop Publishing"	o_7	"Publishing has stopped"
i_8	"Give me a WiFi Name"	o_8	"Press any button"
i_9	"Name Received"	o_{10}	"Publishing has started again"
i_{10}	"Start Publishing"	o_{12}	Device SSID

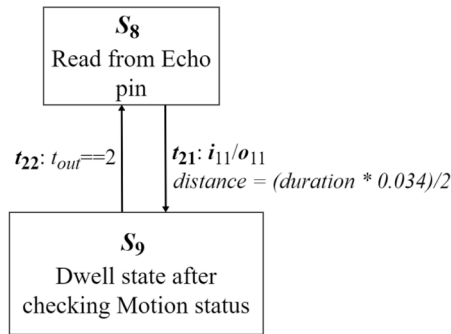


Fig. 4. Part of ultrasonic sensor TFSM. The other part is similar to the temperature sensor TFSM given in Fig. 2.

reading from the temperature sensor. In that state, the input is the temperature reading i_{11} and the microcontroller then outputs the message o_{11} = "Temp Received" and moves to a dwell state S_9 before moving back to state S_8 after a timeout period of 2 seconds. If the temperature sensor does not provide, for any reason, a reading in state S_8 , then the system moves back to state S_5 to check for any messages. As shown in Fig. 2, each dwell state has a timeout period that allows the system to return to another state so that it is never stuck in a lock state.

Motion sensor TFSM: The motion sensor used in this paper is a Passive InfraRed motion sensor (PIR) that detects and measures the heat or infrared light in its surroundings to determine if an object is moving. States S_1 to S_7 , which are used to model the behavior of the microcontroller, are the same between all the modeled sensors. The difference lies in the sensors' functionality shown in states S_8 and S_9 . In state S_8 , the microcontroller checks for a motion signal from the PIR sensor. If there is no motion within the 2 second timeout period, then the system returns to state S_5 . If motion is detected, then the system receives the input i_{11} = MotionStatus(1) from the sensor and publishes the output message o_{11} = "Motion has been detected" and then moves into the dwell state S_9 . Fig. 3 shows the modified states in the motion sensor's TFSM model.

Ultrasonic motion sensor TFSM: The ultrasonic motion sensor detects the distance to nearby objects by emitting a high-frequency sound wave and measuring the time it takes for the echo to return. It contains two parts: a transmitter and a receiver. The transmitter transmits a sound wave with a high ultrasonic frequency and the receiver captures the returning echo. By calculating the time difference between the transmission of the sound and the reception of the echo, the sensor determines the approximate distance to the object. Fig. 4 shows the TFSM of the ultrasonic motion sensor. When the sensor goes into the read state S_8 , it reads the

Table 2
Non-common inputs and outputs of the TFSMs (sensors).

Temperature Sensor			
Input	Description	Output	Description
i_{11}	tempPin(volt)	o_{11}	"Temp Received"
Motion Sensor			
Input	Description	Output	Description
i_{11}	MotionStatus(1)	o_{11}	"Motion has been Detected"
Ultrasonic Motion Sensor			
Input	Description	Output	Description
i_{11}	t received (duration)	o_{11}	"Distance Computed"
RFID Card Reader			
Input	Description	Output	Description
i_{11}	"Data_byte(rchar)"	o_{11}	"Null"
i_{12}	"\$OD"	o_{13}	"Card rejected"
		o_{14}	"Start byte received"
		o_{15}	"Card is reading data bytes"
		o_{16}	"Card accepted"

value from the echo pin $i_{11} = \text{echoPin}(\text{duration})$ and then calculates the distance using the update $\text{distance} = (\text{duration} * 0.034) / 2$. The microcontroller outputs the distance in o_{11} and moves to the dwell state in S_9 .

RFID card reader TFSM: An RFID is a type of wireless communication that uses electromagnetic waves for identification applications. The communication starts when an RFID tag comes near the magnetic field of an RFID card reader and the tag's unique ID is then sent to the microcontroller. The machine modeled for the RFID card reader is shown in Fig. 5. The model assumes that the 10-digit ID is already known by the microcontroller and is represented as a constant string (named "card") in the state diagram. The integer variable "counter" (initially set to zero in the initial state) is used to count the number of digits read. We use the machine to determine whether the read ID value (saved in string variable "code") matches "card" or not. In state S_8 , the microcontroller waits for an RFID tag to come close to the reader. If the input $i_{12} = "0\$D"$, which represents the first received byte, arrives, the machine moves to state S_9 indicating that the start byte has been received (through output o_{14}). As long as $i_{11} = \text{Data_byte}(\text{rchar})$ is received in S_9 , as specified by t_{23} , the machine continues to read the digits, provided that the number of digits is less than 10. It appends the received character "rchar" to the variable "code", increments the "counter" and outputs the message $o_{15} = \text{"Card is reading data bytes"}$. Once the number of digits read reaches 10, as specified by transition t_{24} , the machine moves to state S_{10} to wait for the stop byte, resets the counter to 0 and outputs the message o_{15} . Subsequently, the machine transitions back to state S_8 with different output messages depending on whether the read ID (stored in "code") matches the stored value in ("card"). In all cases, the counter is reset to 0 and the machine returns to state S_8 .

5. IoT attacks

In this section, we describe the IoT security attacks considered in this paper and demonstrate how they are modeled as mutants of the considered TFSMs. By introducing controlled modifications to the original state machines, we capture the anomalous behaviors associated with each attack.

5.1. Battery draining attack

The first attack type considered is the battery draining attack. IoT devices have relatively small batteries and one of the biggest concerns is to ensure that energy consumption cannot be maliciously used to drain the battery quickly. Battery draining attacks achieve this by continuously sending messages that the device is forced to respond to, significantly increasing its energy consumption [44]. We modeled this attack by decreasing the timeout value at each of the states, forcing the machine to leave the state early in order to decrease the probability of the user receiving the correct and expected response. This prompts them to repeatedly resend their input or request until they receive the expected output. As a result, the device undergoes excessive processing, leading to accelerated battery depletion. In this case, taking the motion sensor as an example, the mutants are derived from the motion sensor TFSM by decreasing the timeout period from 2 seconds to 1 second at transitions t_{16} , t_{18} , t_{19} , t_{20} and t_{22} . This is also applied in the temperature sensor, ultrasonic motion sensor and RFID card reader, resulting in 5 mutants for each sensor.

Another type of battery draining attack is sleep deprivation, which prevents the device from entering its low-power sleep state. Unlike other battery-draining attacks that trigger high-energy operations, sleep deprivation attacks keep the device continuously

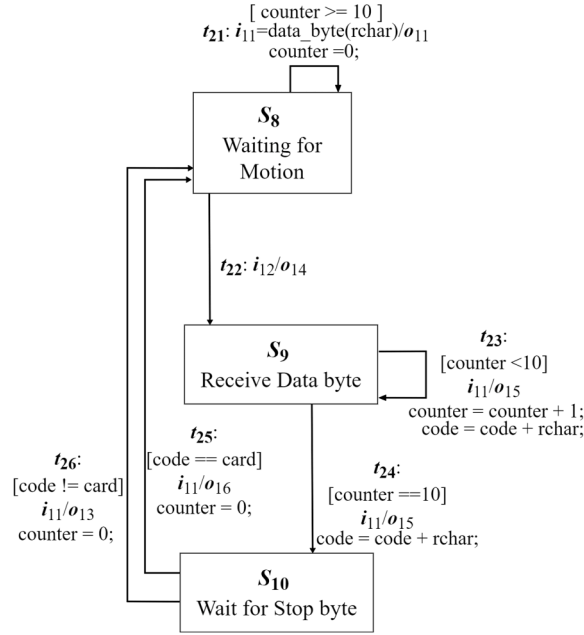


Fig. 5. Part of RFID card reader TFSM. The other part is similar to the temperature sensor TFSM given in Fig. 2.

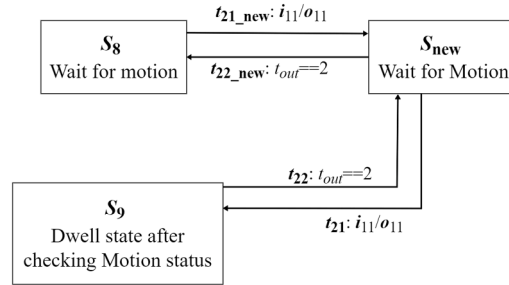


Fig. 6. Modeling Man-in-the-middle attack.

active, preventing it from conserving power. For example, in the motion sensor, the sleep state is S_9 , where the device wakes up every 2 seconds to check if there is any motion. In the corresponding mutants, this is modeled by decreasing the timeout at t_{20} from 2 to 0.001 seconds. Similarly, this is applied to the other three sensors, resulting in 1 additional mutant per sensor for this category.

5.2. Replay attack

Replay attack is the process of eavesdropping on network communication and then replaying the captured data after a period of time [45]. The packet sent is valid but the intent is usually to gain unauthorized access through misdirecting the receiver into thinking that they are the legitimate users. This attack is modeled by replaying t_8 instead of playing t_9 . Therefore, once in S_5 , if the input i_4 is received, o_4 is sent as the output instead of o_6 . This mutant is similarly modeled in the temperature, motion and RFID sensors, resulting in 1 mutant for each sensor.

5.3. Man-in-the-middle attack

The MITM attack is a form of active eavesdropping in which a malicious user places themselves between the legitimate user and the device to intercept and replay messages on their behalf through impersonation [46]. This mutant is created by adding another state between S_8 and S_9 so that when motion is detected, the machine moves to a dwell state similar to S_8 , as shown in Fig. 6. In this case, if motion is not detected again within 2 seconds, the machine is prevented from going to S_9 . This attack can be implemented at all states, but it is implemented at S_8 because it is the most relevant to the functionality of the sensor. This results in 1 mutant for every sensor.

Table 3
Attributes of various data sets.

	Training Features		Contains the studied attacks
	Packet Inter-Arrival Time	Packet payload size	
TON_IoT [21]	No	No	No
BoT-IoT [24]	No	No	No
IoT_23 [22]	Yes	Yes	No
MQTT-IoT- IDS2020 [23]	Yes	Yes	No
IoT Network Intrusion [19]	Yes	Yes	No

5.4. Data falsification attack

A data falsification attack occurs when an attacker injects malicious code into a device, causing it to operate in a manner that deviates from its intended behavior [47]. This attack is modeled by changing the outputs at specific states to the error message o_6 . For example, in S_5 in the motion sensor, instead of outputting o_7 at transition t_{12} , the related mutant would output o_6 . This is applied to transitions $t_8, t_{12}, t_{14}, t_{15}, t_{17}, t_{21}$. This results in 6 mutants for each of the motion sensor, ultrasonic motion sensor and temperature sensor and 10 mutants for the RFID card reader.

6. Classification of IoT attacks

After modeling the devices' normal behavior and their behavior under attack, detecting the attack becomes a classification problem – determining whether incoming IoT data is normal or corresponds to a specific type of IoT attack. This section explains how the test cases for classifying the considered attacks are derived using our proposed formal approach. Additionally, we describe the experimental setup, including the system architecture, the feature extraction methods applied, and the classification models used.

To fairly evaluate our approach, we need a suitable baseline for comparison. We initially considered using external datasets for this purpose; however, any datasets we use will have to contain the two training features we use to train our models: the packet inter-arrival time (IAT) and the packet payloads (this is mentioned in detail in Section 6.3.1). In addition, for our baseline comparison to be meaningful, the dataset must include all four attack types under evaluation; otherwise, the results would not be directly comparable. A dataset containing only one or two of the attacks cannot provide a fair or complete baseline. Since the available external datasets fail to satisfy the two criteria required for a meaningful evaluation, they cannot be used in our study. Some datasets, such as IoTID20 [19] and BoTIoT [24], do not consider the key features we use for training – detailed in Section 6.3 – including the IAT and payload sizes. Other datasets, such as the MQTT-IoT-IDS2020 [23] and IoT_23 [22], appear more relevant as they focus on IoT devices using the MQTT communication protocol, which aligns with our setup. However, as shown in Table 3, these datasets lack the key training features used by our approach and do not consider most of the attack types that we classify in this work – MITM, data falsification, replay, and battery draining (DoS). The IoT_23 [22] and IoT Network Intrusion [19] datasets only contain variations of DDOS attacks, which could be considered a type of battery draining attack, while the MQTT-IoT-IDS2020 [23] contains none of the attacks relevant to our study. As a result, they do not serve as a suitable benchmark for our study. Furthermore, all of the external datasets considered in Kang et al. [19], Alsaedi et al. [21], Garcia et al. [22], Hindy et al. [23], Koroniotis et al. [24] were collected from a different set of IoT devices than those used in our experiment. Since attack behaviors can vary significantly across device types and network setups, evaluating our approach on a dataset with a fundamentally different IoT ecosystem would not yield reliable insights. Due to these limitations, no existing dataset could provide a fair and meaningful comparison. Consequently, we designed our own baseline method to ensure a fair comparison with our formal approach.

Thus, in this section, we present the methodology used to generate both datasets: one based on our proposed formal approach and the other serving as the baseline. Following this, we outline the feature selection criteria for classification and describe the models used in the classification process.

6.1. Formal approach

6.1.1. Experimental setup

Fig. 7 shows the experimental setup of both, the formal and baseline methods. The TFSM representation of each device (normal behavior) is implemented in C++ using the Arduino platform. This is run on an ESP32 microcontroller to model the behavior of the IoT device and is connected to an MQTT broker using WiFi. The messages sent to the microcontroller are injected using middleware, which is implemented using Node-Red. The messages sent and received (tests and their outputs) by the MQTT broker are recorded using WireShark. Mosquitto is used to deploy the MQTT broker.

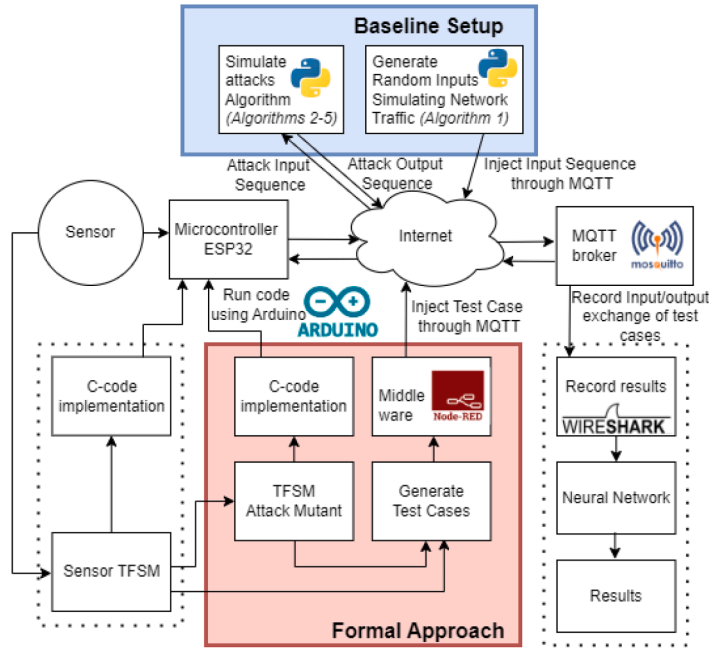


Fig. 7. Experimental setup architecture.

6.1.2. Test derivation using the formal approach

For each IoT device considered, mutation testing is used to derive test cases that distinguish the TFSM model of the device from the corresponding mutants that represent the attacks considered. In our case, these test cases are derived manually as the number of these test cases is very small. However, in general, for large number of mutants, one can automatically derive these tests using the methods described in Bresolin et al. [11], Petrenko et al. [12], Simão et al. [15], Sugeta et al. [16], Pinheiro et al. [17], Tvardovskii et al. [18].

6.1.3. Mutant deployment

The normal behavior of the sensors are modeled as TFSMs, which are then manually written as C++ code. The code mutants, representing the modeled attacks (device behavior under various network attacks), are then generated by changing the C++ code. Each code mutant is also deployed using Arduino which uploads the C++ implementation onto the ESP32 microcontroller. Each deployed implementation is thoroughly tested using randomly derived test cases and the test cases explained in the following subsection.

6.1.4. Test case execution and data collection

For each TFSM mutant representing an attack, a test case is created that distinguishes that TFSM mutant from the related TFSM normal device behavior. The test case is injected using Node-Red, which publishes the test case input onto a topic that the microcontroller is subscribed to. For all considered sensors, the device connects to MQTT in S_4 and moves to S_5 . After connecting to the MQTT broker, the inputs of the test case are injected using Node-Red, which is then also used to monitor the output messages of the device in response to the injected test case inputs. Then for data collection, Wireshark is used to capture the communication exchange (test case inputs and corresponding outputs) on the network by deploying it on the device running the MQTT broker. For more accurate classification results, we collect 500 samples per attack in the form of pcap files. In other words, each test case is run many times so that for each attack the total number of related collected samples becomes 500.

6.2. Baseline method

To fairly evaluate our approach, we developed a traditional baseline method that does not rely on FSM-based modeling. In designing the baseline, we ensure that its data collection process closely follows the methodologies used in existing datasets, allowing for a meaningful comparison between our formal approach and a conventional data collection strategy.

6.2.1. Experimental setup

For the baseline method, only the C++ implementation of the normal behavior of the sensors is deployed using Arduino onto the ESP32 microcontroller. Randomized inputs are generated using a Python script (shown in Algorithm 1) and are published through MQTT. The messages are received by the ESP32 and it sends a corresponding output message through MQTT in accordance with the model's behavior.

The attacks are simulated externally on the resulting network traffic through the use of an attack algorithm run on a "second PC". The "second PC" runs the input generator script and the attack algorithm, while the first PC is used to host the MQTT server. There are 4 different attack algorithms used, one for each network attack category, and each attack algorithm targets a specific part of the network traffic (the attack input sequence) and modifies the traffic depending on the attack (attack output sequence). The attack algorithms are provided in Algorithms 2 - 5 given below. The attack algorithms we designed and implemented in the baseline method were developed to replicate those found in the literature while adapting them to our own architecture and IoT devices. For simplicity, the specific network packets targeted by an attack algorithm is called the *attack input sequence* and the change performed by the attack on the network traffic is called the *attack output sequence*.

6.2.2. Training tests derived using baseline technique

In the baseline method, instead of modeling the behavior of the device under attack, we externally simulate the attacks using a python script while using the C++ implementation of the normal behavior of the sensor. For all simulated attacks, we use a "second PC" that runs the Python script, which randomly sends inputs to the IoT system through MQTT, as shown in Algorithm 1, with randomly chosen delays of values of 0, 1 or 2 seconds. The script waits for the system to connect to the MQTT server before injecting the random inputs through the publish feature of MQTT. At the same time, another Python script is run on the "second PC" which performs the attack on the network traffic. The final network traffic resulting from the input generator, the device's response to the inputs, and the change from the attack algorithm will result in a test case that is recorded using Wireshark for later model training. The total execution time of each test case ranges from 10 to 14 seconds. For each attack, this process is repeated 500 times to obtain a sufficient number of test cases for training the DL models on that attack. Note that since the test cases contain randomized inputs, each test case is considered unique. Wireshark is deployed on the first PC hosting the MQTT broker to record the test cases.

Algorithm 1: Generation of random inputs using baseline method.

Connect to MQTT

```
function testcase():
    while(total-time < 10):
        var payload = random.choice(input-options)
        publish(topic, payload)
        var delay = random.choice(delay-options)
        time.sleep(delay)
        total-time = total-time + delay

while True:
    if(message == "Connected to MQTT"):
        testcase()
```

Battery draining attack simulation algorithm:

Battery draining attacks usually take the form of denial of service (DoS) attacks [48,49] and is usually performed by flooding the target device, but as we are using MQTT as the communication protocol between the devices, the target of the attack should be the MQTT server. This can be performed by repeatedly ping the server IP address [48,49] or by repeatedly sending the server CONNECT requests until it crashes [50,51]. Another way is to simply continuously publish messages to the server until the buffer is completely full and the server either crashes or is unable to receive and send messages. Therefore, the battery draining attack algorithm monitors packets received by the "second PC" and for every packet, 50 messages are published with randomly chosen inputs as the payload, with a 1 millisecond delay between each. This floods the server and causes service to stop within 1 minute. The python script for the DoS attack is also deployed on the "second PC" and related details are provided in Algorithm 2. The attack input sequence is the microcontroller output response to the python script in Algorithm 1, and the attack output sequence are the packets used for flooding the MQTT server.

Man-in-the-Middle attack simulation algorithm: For the MITM attack, the same setup is used, with a second machine running a Python script that generates and publishes random inputs, as shown in Algorithm 1. The MITM attack is based on the idea that data is being monitored and manipulated in between two legitimate users. It is an active attack that either delays the packet or modifies it. Accordingly, we deploy on the first PC, that contains the MQTT server, a python script that utilizes the pydivert tool to perform 3 forms of MITM attacks. Pydivert is a python wrapper for the tool WinDivert, which is a user-mode sniffing/modification package for windows systems. It monitors packets being sent and received by the PC and filters the packets sent to and from the windows network stack according to a set of conditions, such as source and destination IP addresses and port numbers. Any packets that comply with the conditions are intercepted for modification before being re-injected into the network stream. We set the tool to filter packets sent from the ESP32 to the MQTT server to perform MITM attacks on the output messages of the microcontroller. The first of the three MITM attacks is a simple delay, where the packet is held for a random time between 1 and 3 seconds before being re-injected into the network stream. The second MITM attack considers modifying the packet payload by shuffling the payload, and the third also modifies the packet payload by erasing some characters. In this case, the attack input sequence is the microcontroller's output

Algorithm 2: Simulated battery draining attack.

```

Connect to MQTT

function testcase():
for i in range(3):
    var payload = random.choice(input-options)
    publish(topic, payload)

while True:
if(message received):
    if(message == "Connected to MQTT):
        testcase()
    else:
        while counter < 50 :
            var payload = random.choice(input-options)
            publish(topic, payload)
            time.sleep(1)

```

response messages and the attack output sequence is the modified packets. [Algorithm 3](#) includes the pseudocode of the MITM attack algorithm.

Algorithm 3: Simulated man-in-the-middle attack.

```

Connect to MQTT

while True:
if (packet.srcAddr = ESP32.ipAddr and packet.destAddr = MQTTServer.ipAddr):
    var delay = random.choice(delay-options)
    time.sleep(delay)
    //OR shuffle(packet.payload)
    send(packet)

```

Replay attack simulation algorithm: A replay attack is performed by monitoring the packets sent to the microcontroller from [algorithm 1](#) and then replaying them after a certain period of time. The attack algorithm is deployed on the second PC, and it is simulated by using the pydivert tool to monitor the inputs being sent to the MQTT server and send them again after a random time chosen between 1 and 3 seconds. Accordingly, the attack input sequence is the input messages sent to the microcontroller, and the attack output sequence is the replayed duplicate messages sent to the microcontroller. [Algorithm 4](#) includes the pseudocode of the replay attack algorithm.

Algorithm 4: Simulated replay attack.

```

Connect to MQTT

function replay(payload):
    var delay = random.choice(delay-options)
    time.sleep(delay)
    publish(topic, payload)

while True:
if (packet.destAddr = ESP32.ipAddr and packet.srcAddr = MQTTServer.ipAddr):
    var message = packet.payload
    send(packet)
    new Thread( replay(message) )

```

Data Falsification attack simulation algorithm: A data falsification attack occurs when a malicious entity injects code into the system to produce false outputs. When simulating the data falsification attack, only packets that include sensitive or essential information are modified. As the main information sent by the microcontroller is the sensor reading, we only consider packets containing the sensor information for this attack. Thus, we deploy the attack algorithm on the "second PC" to capture the random inputs being

sent to the microcontroller and if a packet containing the sensor information is published, the input is modified and re-injected into the network stream. Taking the temperature sensor as an example, when a packet containing the recorded temperature is sent, the python script modifies the reading by adding to it or subtracting from it a random number between -100 and 100°C. [Algorithm 5](#) includes the pseudocode of the data falsification attack algorithm.

Algorithm 5: Simulated data falsification attack.

```

Connect to MQTT

while True:
    if (packet.destAddr == ESP32.ipAddr and packet.srcAddr == MQTTServer.ipAddr):
        var message = packet.payload
        (if message == temperature-reading):
            packet.payload = message ± random.uniform(-100,100)
            send(packet)

```

6.3. Building the NN models

In this section, we describe the features extracted from the Wireshark PCAP files and used for training, as well as the architecture of the DL models used for classification. We train and test the models using an RTX 4000 GPU.

6.3.1. Feature extraction

To train our DL model, the PCAP files need to be parsed. First, the packets are filtered based on network protocol, where we only keep the MQTT packets.

The two features we use to train our models are the inter-arrival time and the packet payload size. We use the IAT feature for attack classification as it is one of the most valuable and effective network traffic features that provides insight into the timing patterns of packet transmissions, which makes it particularly useful for detecting anomalous behaviors in IoT networks. In a previous study [52], the authors conducted a comprehensive analysis of multiple well-known IDS datasets to identify the most impactful features for intrusion detection. Their findings highlighted that IAT consistently ranked among the most informative features across the different datasets, demonstrating its effectiveness in distinguishing normal traffic from attack patterns.

The IAT is extracted from the PCAP files using tshark, a network traffic analyzer. The length of the resulting IAT array depends on the number of inputs and outputs in the test case trace. To compare multiple arrays, each IAT value must correspond to the same time offset from the start of the test case. For example, the fourth entry in every array represents the value at 0.5 seconds. To achieve this alignment, we plot the arrays and apply interpolation to obtain 8 evenly spaced points per second, producing 48 IAT values for each test case (using the first 6 seconds of each PCAP file). If the total time of a test case is longer than 6 seconds, any value after the 6-second mark is discarded, and if the total time is shorter than that, the IAT array is padded with zeros to fill in the full 48 values. This allows us to determine if the IDS system could determine the attack from a 6 second window of collected network traffic. In addition, the payload lengths are also extracted to record the difference in response to the injected test case. The first 4 packets are CONNECT and ACK packets, while the rest are the requests and responses sent and received by the microcontroller. Each packet payload size is stored multiple times in the payload array. If the test case results in a payload array having more than 48 values, the rest of the values are discarded; otherwise, the array is again padded with zeros. As the lengths of the test cases for the first three sensors (the temperature sensor, motion sensor and ultrasonic motion sensor) were short, the resulting array is padded to reach the size of 48. However, for the RFID card reader, the length of each resulting test case is larger than 12 for the payload size of each packet to be stored multiple times. This would result in loss of information, and so each packet payload size was inserted once in the array to ensure that the payload length of all the packets in the test case can be accommodated in an array of size 48. This is applied to both the baseline and the formal methods for accurate comparison. The two feature arrays, IAT and payload size, were normalized separately and concatenated.

6.3.2. LSTM architecture

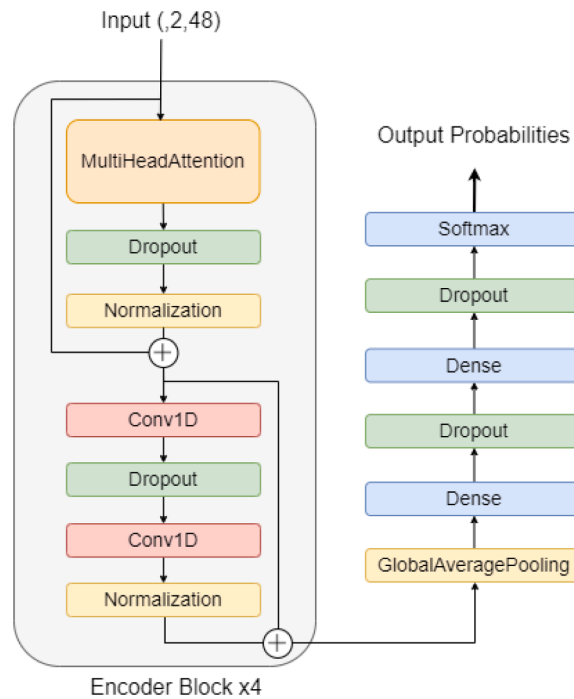
The LSTM model generally performs well with the classification of network data in the IDS domain, as mentioned in [Section 2](#). The model we use contains 1 LSTM layer with 256 nodes and a dropout layer to prevent overfitting, followed by a dense layer with the number of output nodes equal to the number of classes. Our formal method performs multi-class classification on the following classes: Normal behavior, MITM attack, replay attack, battery draining attack and data falsification attack. While the baseline method performs multi-class classification on the 4 attack classes only. Therefore, the formal method has 5 nodes in the output layer while the baseline contains 4 nodes for the total number of classes. A description of the LSTM model layers is shown in [Table 4](#).

6.3.3. Transformer architecture

We deploy a transformer model that is composed of an encoder-feed forward network structure. The transformer encoder consists of multiple identical layers, each containing a multi-head self-attention layer and 1D convolution layers since our input data is one-dimensional. The multi-head self-attention mechanism is the core of the transformer, which allows the model to focus on different

Table 4
LSTM architecture.

Layer	Neurons	Activation Function
LSTM	256	–
Dropout	(0.4)	–
Dense	5	Softmax

**Fig. 8.** Transformer architecture.**Table 5**
Encoder block layer characteristics.

Layer	Neurons	Filter	Activation Function
MultiHeadAttention	256 x 8	–	–
Dropout	(0.25)	–	–
LayerNormalization	–	–	–
Conv1D	–	32	relu
Dropout	(0.25)	–	–
Conv1D	–	48	–
LayerNormalization	–	–	–

parts of the input sequence simultaneously to capture various dependencies. The encoder output tensor is reduced to a vector of features using a pooling layer and appended by multiple multilayer perceptron units. A detailed view of the architecture is shown in Fig. 8, where we used four encoder blocks and two multi-layer perceptron units. A description of the encoder and transformer layers are shown in Tables 5 and 6, respectively.

6.3.4. Model training

We evaluate the models using 10-fold stratified cross-validation, repeated three times for each model. This results in a total of 30 folds collected for each model under each experimental condition. For each fold, we split the data set into training, testing and validation sets, where the validation set is 15% of the training set. The model's input has a shape of (2, 48), where the first dimension is the IAT values array and the second is the packet payload size array. We use sparse categorical cross entropy as the loss function and an Adam optimizer with a learning rate of 0.001. We run the models with 300 epochs and a batch size of 32, details shown in Table 7. Both models were trained and evaluated on the same hardware (NVIDIA RTX 4000 GPU). Under identical conditions, the LSTM model required approximately 60 minutes for training, whereas the transformer-based model required approximately 180 minutes.

Table 6
Transformer layer characteristics.

Layer	Neurons	Filter	Activation Function
Encoder Block [x4]	–	–	–
AveragePooling1D	–	–	–
Dense	256	–	relu
Dropout	(0.4)	–	–
Dense	128	–	relu
Dropout	(0.4)	–	–
Dense	5	–	softmax

Table 7
Training hyperparameters for the LSTM and transformer models.

Hyperparameter	LSTM	Transformer
Learning rate	0.001	0.001
Optimizer	Adam	Adam
Batch size	32	32
Epochs	300	300

Table 8
Results of LSTM model when trained on temperature sensor data derived using our formal approach and baseline approach.

	Baseline Method				Formal Method			
	F1-score	Precision	Recall	Accuracy	F1-score	Precision	Recall	Accuracy
Mean	0.70360	0.7098	0.7075	0.7075	0.9948	0.9947	0.9950	0.9944
Median	0.7037	0.7091	0.705	0.705	0.9942	0.9948	0.9947	0.9933
Max	0.7481	0.7589	0.75	0.75	1.0	1.0	1.0	1.0
Min	0.6517	0.6611	0.65	0.65	0.9871	0.9867	0.9877	0.9867
Std. Dev	0.02607	0.02582	0.02548	0.02548	0.003710	0.003874	0.003562	0.004029

Table 9
Results of transformer model when trained on temperature sensor data.

	Baseline Method				Formal Method			
	F1-score	Precision	Recall	Accuracy	F1-score	Precision	Recall	Accuracy
Mean	0.7271	0.7536	0.7523	0.7523	0.9863	0.9869	0.9870	0.9845
Median	0.7306	0.7612	0.75	0.75	0.9942	0.9935	0.9951	0.9933
Max	0.8036	0.8165	0.805	0.805	1.0	1.0	1.0	1.0
Min	0.6422	0.6085	0.63	0.63	0.9466	0.9498	0.9382	0.94
Std. Dev	0.04411	0.04935	0.03274	0.03274	0.01614	0.01495	0.01572	0.01834

7. Experimental evaluation

In this section, we include the results obtained by training the considered neural network models using the collected data. We note that all the artifacts related to the work presented in this paper are available in <https://bit.ly/4e4Lv72>

7.1. Evaluation criteria

We evaluate the models using four performance metrics: accuracy, recall, precision, and F1-score. The equations for the chosen metrics are given below in (1)–(4) given True Negatives (TN), True Positives (TP), False Negatives (FN) and False Positives (FP).

$$Accuracy = \frac{(TP + TN)}{All} \quad (1)$$

$$Recall = \frac{TP}{(TP + FN)} \quad (2)$$

$$Precision = \frac{TP}{(TP + FP)} \quad (3)$$

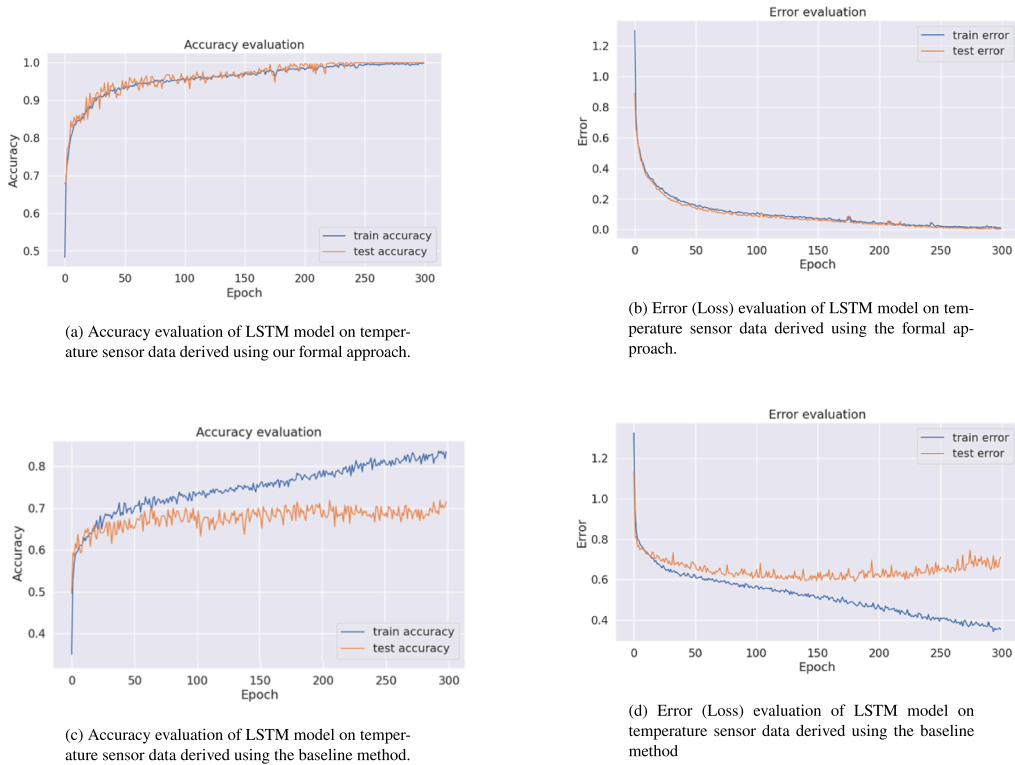


Fig. 9. Evaluation graphs for LSTM model on temperature sensor data.

$$F1 - score = \frac{2 \times Recall \times Precision}{(Recall + Precision)} \quad (4)$$

We plot the training and validation accuracy and loss curves versus the number of epochs. This allows us to observe whether the trained models are overfitting the data. In addition, for each considered evaluation metric, the Kruskal Wallis test [53] is used to determine whether the distributions of the results of evaluation metrics differ significantly between models, where a difference is considered significant when the p-value is below 0.05.

7.2. Results

In the following section, we provide the detailed results of the temperature sensor. Similar results are observed for the other considered sensors. These results are summarized at the end of this section; however, due to space limitations, the details of these results are provided in <https://bit.ly/4e4Lv72>. For simplicity, the data derived using the formal approach will hereafter be referred to as formal data, while the data obtained using the baseline approach will be referred to as baseline data.

LSTM results over formal and baseline data: Table 8 shows the results of the average F1-score, precision, recall and accuracy of the LSTM model when trained on the formal and baseline data. And for each evaluation metric, Table 10 (Column I) shows the corresponding Kruskal Wallis test results over the 30 pairs of runs of the LSTM model over the formal and baseline data, respectively. According to the obtained results:

- For each considered metric, a significant difference is observed between the LSTM models trained using the formal data and baseline data. Specifically, the LSTM model trained on the formal data significantly outperforms that trained on the baseline data in attack classification across all considered evaluation metrics. The average F1-score of the LSTM model when trained on the formal data is 99% (ranging from 98% to 100%) while it is 70% (ranging from 65%-74%) for the LSTM trained on the baseline data.

Fig. 9a and b plot the accuracy and error (loss) curves of the LSTM when trained on the formal data. Fig. 9c and d plot the accuracy and error (loss) curves of the LSTM when trained on the baseline data. According to these figures:

- The LSTM trained on the formal data shows a sharp initial increase in accuracy to about 80% and a steady improvement afterwards. Since the training and testing accuracies increase simultaneously, this indicates that the model is not overfitting during training. However, the LSTM trained on the baseline data shows that the model is overfitting on the training data, as the train accuracy increases continuously to approximately 80%, while the test accuracy stops at 65%, with the two curves diverging at epoch 25.

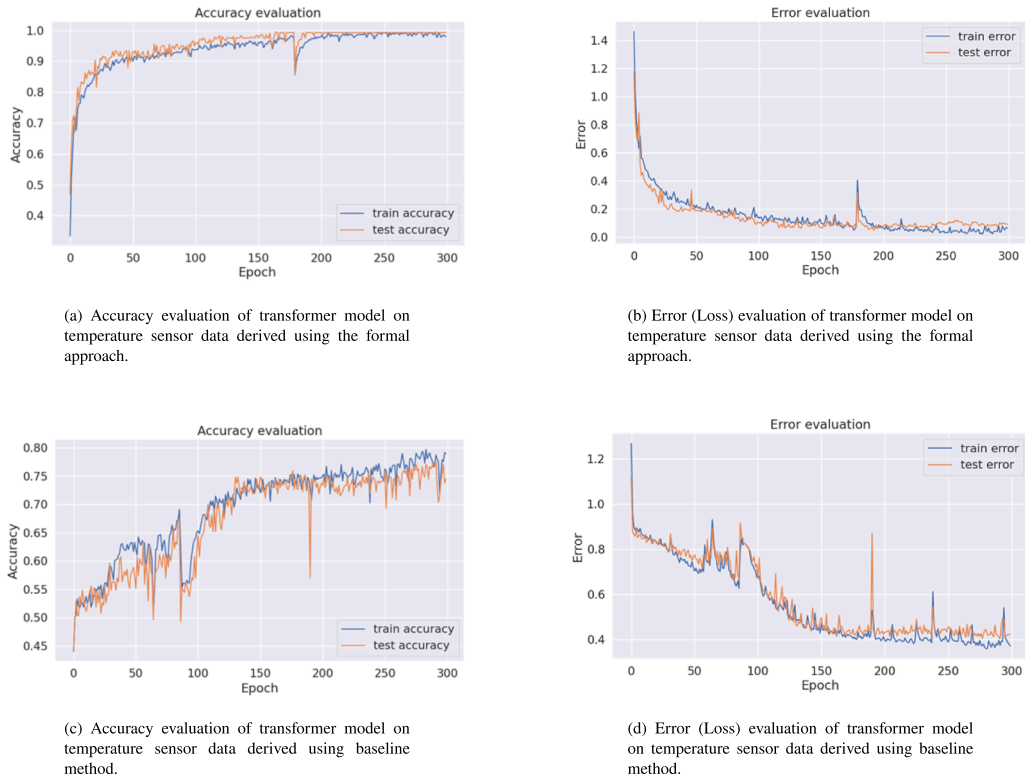


Fig. 10. Evaluation graphs for transformer model on temperature sensor data.

Table 10

Column I (II) contains the average of the statistical comparison of the various metrics (Rows) between LSTM (transformer) models trained on data derived using our formal approach and the baseline method.

	LSTM (I)	Transformer (II)
	p-value	p-value
F1-Score	2.790e-11	2.818e-11
Precision	2.734e-11	2.811e-11
Recall	2.765e-11	2.739e-11
Accuracy	2.340e-11	2.543e-11

Table 11

Statistical comparison of various metrics between the transformer and LSTM models trained on the formal approach data.

	Temperature	Motion	Ultrasonic Motion	RFID
	p-value	p-value	p-value	p-value
F1-Score	0.272	0.056	0.108	0.715
Precision	0.217	0.056	0.182	0.777
Recall	0.281	0.048	0.047	0.754
Accuracy	0.206	0.053	0.165	0.969

Transformer models results over formal and baseline data: Table 9 presents the results of the average F1-score, precision, recall, and accuracy for both the formal and baseline data runs using the transformer models. For each evaluation metric, Table 10 (Column II) shows the corresponding Kruskal Wallis test results over the 30 pairs of runs of the transformer model over the formal and baseline data, respectively. According to these results:

- For each considered metric, there is a significant difference between the transformer model trained on the formal and baseline data. For each considered evaluation metric, the transformer model trained on formal data significantly outperforms that trained

Table 12

Percentage gain in the average F1-scores between the models trained on the formal and baseline data for the LSTM and transformer models.

IoT Device	Percentage gain of LSTM	Percentage gain of Transformer
Temperature	41.387%	35.648%
Motion	49.217%	67.472%
Ultrasonic Motion	53.312%	60.986%
RFID	63.412%	77.130%

on the baseline data in attack classification. The average F1-score of the transformer model when trained on the formal data is 98% (ranging from 94% to 100%) while it is 72% (ranging from 64%–80%) for the transformer trained on the baseline data.

Fig. 10a and b plot the accuracy and error (loss) curves of the transformer when trained on the formal data while Fig. 10c and d plot the accuracy and error (loss) curves of the transformer model when trained on the baseline data. According to these figures:

- The transformer model trained on formal data initially exhibit a sharp increase in accuracy, reaching approximately 90%, followed by a steady improvement. In addition, the graphs indicate that the proposed approach does not overfit the training data. Notably, the accuracy and error curves show that the LSTM model trained on the baseline data results in a higher degree of overfitting on the training data compared to the transformer model trained on the baseline data. It is also worth noting that the graphs for the transformer models display greater fluctuations and less stability than those of the LSTM models.

Other results: According to the results depicted in Table 11, the Kruskal Wallis test was conducted on the results of the LSTM and transformer models when trained using the formal data. The results reveal that:

- For each evaluation metric, there are no significant differences between the results of the LSTM and transformer models when trained on the formal data.

Results over other sensors: The remaining sensors exhibit similar results, where the LSTM model trained on the formal data consistently outperforms the one trained on the baseline data. As shown in column I of Table 12, the percentage gain in the average F1-scores of across all sensors demonstrates the improved classification performance of the LSTM model achieved using the formal data. Similarly, the transformer model trained on the formal data consistently achieves higher performance than the one trained on the baseline data. Column II of Table 12 highlights the percentage gain in average F1-scores across all sensors, demonstrating the enhanced classification performance of the transformer model obtained when trained on the formal data.

Results over number of derived tests: It is worth mentioning that not only the results of the formal model outperform those of the baseline model across all evaluation metrics; but also the number of tests derived by the formal approach is significantly less than those derived by the baseline approach. For instance, the number of tests derived for the temperature sensor attacks using the formal approach is only 10 while this number is 2000 for the baseline approach. On average, over all considered experiments, the number of test cases derived by the formal approach is 95.5% less than those of the baseline approach. For a detailed summary of these tests the reader may refer to "TestCase Comparison" folder in the <https://bit.ly/4e4Lv72>

8. Conclusion

We propose a formal approach for intrusion detection systems using deep learning models trained on data derived using formal modeling and mutation testing. We model four common lightweight sensors as timed state machines, and then we appropriately mutate these machines to model the common IoT security attacks that we intend to classify; namely, the man-in-the-middle, replay, data falsification, and battery-draining attacks. Mutation testing is then used to derive tests that distinguish the machines from the attacks. These tests are used as a concise data set to train learning models instead of using conventionally generated data. The corresponding network data from the setup is recorded, parsed, and used to train long short-term memory and transformer deep learning models to classify the different attacks. In addition, we also propose a baseline method that externally simulates IoT network attacks to collect and train the learning models. A framework is implemented for the presented approach. According to the conducted experiments, the results for the proposed formal models are high across the board for all considered evaluation metrics, and these results are significantly better than the baseline trained models; namely, the average percentage gains in the F1-score of the LSTM and transformer models are of 51.8% and 60.3%, respectively. In addition, the number of test cases derived by the formal approach is 95.5% less than those of the baseline approach.

The scalability of the proposed method follows from the fact that each sensor is modeled independently and the analysis for each sensor is performed at the edge independent of the other sensors. In large systems with many homogeneous sensors, modeling a single representative device is sufficient because identical devices exhibit the same behavioral patterns. In systems with heterogeneous devices, each distinct device type is modeled separately, and scalability depends on the behavioral complexity of those devices rather than on the total number of nodes. Most devices would be easily modeled, so anything with predictable, periodic, or event-driven traffic is naturally easy to model such as temperature, motion, RFID readers, light sensors, smart TVs, smart hubs etc. Therefore, our system design would be applicable on smart-home ecosystems (Philips Hue, Samsung SmartThings etc), commercial building automation (KNX building automation systems, BACnet-based HVAC systems such as honeywell building management systems) and

specific industrial IoT systems. Other architectures where our design would be applicable include [54] and [55] as the sensors and actuators use exhibit well-defined finite-state behaviors that can be formally modeled and systematically mutated to generate structured datasets for IDS evaluation. In more complex IoT architectures where sensors communicate through a hub or controller, normal and attack behaviors would need to be defined over the appropriate composition of interacting components rather than a single device. Extending the formal approach to more complex devices and to capture such multi-component IoT systems is a direction for future work. In addition, our evaluation also aligns closely with the dominant classes of IoT network threats identified in prior work [56], giving our results broad relevance across the typical IoT threat landscape. It effectively detects attacks that manifest through timing, ordering, or payload-size deviations, such as replay, MITM, data-falsification, battery-draining, and device-level flooding. These threats frequently arise in deployments of simple, event-driven sensors. Attacks requiring high-bandwidth data, semantic analysis, or coordinated multi-device behavior remain outside the scope of our current model, and extending the framework to cover these additional threat classes—including botnets and other forms of malware—constitutes an important direction for future work.

Another avenue for future work is to broaden the range of IoT devices and incorporate additional security threats, such as botnets and other forms of malware, to increase the coverage and robustness of the detection framework. Furthermore, it would be also useful to extend the proposed threat detection approach to handle diagnosing and localizing the specific mechanisms underlying each detected attack.

CRedit authorship contribution statement

Yara Kaddoura: Writing – original draft, Validation, Software, Methodology, Data curation, Conceptualization; **Khaled El-Fakih:** Writing – original draft, Validation, Supervision, Methodology, Formal analysis, Conceptualization; **Imran Zualkernan:** Writing – original draft, Methodology, Formal analysis, Conceptualization; **Alifiya Bhanpurawala:** Methodology, Formal analysis, Conceptualization.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The first two authors would like to acknowledge the partial support of AUS FRG grant FRG24-C-E-50-M .

References

- [1] A. Rejeb, K. Rejeb, S. Simske, H. Treiblmaier, S. Zailani, The big picture on the Internet of Things and the smart city: a review of what we know and what we need to know, *Internet of Things* 19 (2022) 100565. <https://www.sciencedirect.com/science/article/pii/S2542660522000609>. <https://doi.org/10.1016/j.iot.2022.100565>
- [2] M. Mahmoud, M. Kasem, A. Abdallah, H.S. Kang, AE-LSTM: Autoencoder with LSTM-based intrusion detection in IoT, in: 2022 International Telecommunications Conference (ITC-Egypt), IEEE, 2022, pp. 1–6. Place: Alexandria, Egypt <https://ieeexplore.ieee.org/document/9855688/>. <https://doi.org/10.1109/ITC-Egypt55520.2022.9855688>
- [3] A. Bhanpurawala, K. El-Fakih, I. Zualkernan, A Formal Assisted Approach for Modeling and Testing Security Attacks in IoT Edge Devices, 2022, [arXiv: 2210.05623](https://arxiv.org/abs/2210.05623) [cs].
- [4] D. Swessi, H. Idoudi, A survey on Internet-of-Things security: threats and emerging countermeasures, *Wirel. Pers. Commun.* 124 (2) (2022) 1557–1592. <https://link.springer.com/10.1007/s11277-021-09420-0>. <https://doi.org/10.1007/s11277-021-09420-0>
- [5] J. Deogirikar, A. Vidhate, Security attacks in IoT: a survey, in: 2017 International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC), IEEE, 2017, pp. 32–37. Place: Palladam, Tamilnadu, India, <http://ieeexplore.ieee.org/document/8058363/>. <https://doi.org/10.1109/I-SMAC.2017.8058363>
- [6] D. Harel, A. Naamad, The STATEMATE semantics of statecharts, *ACM Trans. Softw. Eng. Methodol.* 5 (4) (1996) 293–333.
- [7] ITU-T, Recommendation Z.100-Specification and Description Language (SDL), 1994,
- [8] Object Management Group (OMG), Unified Modelling Language v1.4, 2002, (<https://www.omg.org/spec/UML/1.4/About-UML>).
- [9] M.G. Merayo, M. Núñez, I. Rodríguez, Formal testing from timed finite state machines, *Comput. Netw.* 52 (2) (2008) 432–460.
- [10] R.M. Hierons, M. García Merayo, M. Nuñez García, Testing from a stochastic timed system with a fault model, *J. Log. Algebr. Program.* 78 (2) (2009) 98–115.
- [11] D. Bresolin, K. El-Fakih, T. Villa, N. Yevtushenko, Equivalence checking and intersection of deterministic timed finite state machines, *Formal Method. Syst. Des.* 59 (1) (2021) 77–102.
- [12] A. Petrenko, S. Boroday, R. Groz, Confirming configurations in EFSM testing, *IEEE Trans. Softw. Eng.* 30 (1) (2004) 29–42.
- [13] C. Bourhfir, E. Abdoulhamid, K. Khendek, R. R. Dssouli, Test cases selection from SDL specifications, *Comput. Netw.* 35(6) (2001) 693–708.
- [14] A.S. Kalaji, R. Hierons, S. Swift, An integrated search-based approach for automatic testing from extended finite state machine (EFSM) models, *Inf. Softw. Technol.* 53 (2011) 1297–1318.
- [15] A. Simão, A. Ambrósio, S. Fabbri, A. do Amaral, E. Martins, J.C. Maldonado, Plavis/FSM: an environment to integrate FSM-based testing tools, in: Tool Session of XIX Brazilian Symposium on Software Engineering, 2008, pp. 1–6.
- [16] T. Sugeta, J.C. Maldonado, E. Wong, Mutation testing applied to validate SDL specifications, in: Testing of Communicating Systems, 2004, pp. 193–208.
- [17] A.C. Pinheiro, A. da Silva Simão, A.M. Ambrosio, FSM-Based Test case generation methods applied to test the communication software on board the ITASAT university satellite: a case study, *J. Aerosp. Technol. Manag.* 6 (2014) 447–461.

- [18] A. Tvardovskii, K. El-Fakih, N. Yevtushenko, Testing and incremental conformance testing of timed state machines, *Sci. Comput. Program.* 233 (2024) 103053. <https://www.sciencedirect.com/science/article/pii/S0167642323001351>. <https://doi.org/10.1016/j.scico.2023.103053>
- [19] H. Kang, D.H. Ahn, G.M. Lee, J.D. Yoo, K.H. Park, H.K. Kim, IoT network intrusion dataset, 2019, <https://iee-dataport.org/open-access/iot-network-intrusion-dataset>. <https://doi.org/10.21227/Q70P-Q449>
- [20] I. Ullah, Q.H. Mahmoud, A scheme for generating a dataset for anomalous activity detection in IoT networks, in: C. Goutte, X. Zhu (Eds.), *Advances in Artificial Intelligence*, 12109, Springer International Publishing, Cham, 2020, pp. 508–520. http://link.springer.com/10.1007/978-3-030-47358-7_52. https://doi.org/10.1007/978-3-030-47358-7_52
- [21] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, A. Anwar, TON_IoT telemetry dataset: a new generation dataset of IoT and IIoT for data-driven intrusion detection systems, *IEEE Access* 8 (2020) 165130–165150. <https://ieeexplore.ieee.org/document/9189760/>. <https://doi.org/10.1109/ACCESS.2020.3022862>
- [22] S. Garcia, A. Parmisano, M.J. Erquiaga, IoT-23: a labeled dataset with malicious and benign IoT network traffic, 2020, <https://zenodo.org/record/4743746>. <https://doi.org/10.5281/ZENODO.4743746>
- [23] H. Hindy, E. Bayne, M. Bures, R. Atkinson, C. Tachtatzis, X. Bellekens, Machine Learning Based IoT Intrusion Detection System: An MQTT Case Study (MQTT-IoT-IDS2020 Dataset), 2020, [arXiv: 2006.15340 \[cs\]](https://arxiv.org/abs/2006.15340).
- [24] N. Koroniotis, N. Moustafa, E. Sitnikova, B. Turnbull, et al., Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset, 2018, [arXiv:1811.00701 \[cs\]](https://arxiv.org/abs/1811.00701).
- [25] I. Vaccari, G. Chiola, M. Aiello, M. Mongelli, E. Cambiaso, et al., MQTTset, A new dataset for machine learning techniques on MQTT, *Sensors* 20 (22) (2020) 6578. <https://www.mdpi.com/1424-8220/20/22/6578>. <https://doi.org/10.3390/s20226578>
- [26] M.H. Jamal, N. Naz, M.A.K. Khattak, F. Saeed, S.N. Altamimi, S.N. Qasem, A comparison of re-sampling techniques for detection of multi-Step attacks on deep learning models, *IEEE Access* 11 (2023) 127446–127457. <https://doi.org/10.1109/ACCESS.2023.3332512>
- [27] S.W. Azumah, N. Elsayed, V. Adewopo, Z.S. Zaghloul, C. Li, A deep LSTM based approach for intrusion detection IoT devices network in smart home, in: 2021 IEEE 7th World Forum on Internet of Things (WF-IoT), IEEE, 2021, pp. 836–841. Place: New Orleans, LA, USA, <https://ieeexplore.ieee.org/document/9596033/>. <https://doi.org/10.1109/WF-IoT51360.2021.9596033>
- [28] N. Elsayed, Z.S. Zaghloul, S.W. Azumah, C. Li, Intrusion Detection System in Smart Home Network Using Bidirectional LSTM and Convolutional Neural Networks Hybrid Model, 2021, [arXiv: 2105.12096 \[cs\]](https://arxiv.org/abs/2105.12096).
- [29] N. Islam, F. Farhin, I. Sultana, M. Shamim Kaiser, M. Sazzadur Rahman, M. Mahmud, A. S. M. Sanwar Hosen, G. Hwan Cho, et al., Towards machine learning based intrusion detection in IoT networks, *Comput. Mater. Contin.* 69 (2) (2021) 1801–1821. <https://www.techscience.com/cmc/v69n2/43889>. <https://doi.org/10.32604/cmc.2021.018466>
- [30] H. Alkahtani, T.H.H. Aldhyani, Intrusion detection system to advance Internet of Things infrastructure-based deep learning algorithms, *Complexity* 2021 (2021) 1–18. <https://www.hindawi.com/journals/complexity/2021/5579851/>. <https://doi.org/10.1155/2021/5579851>
- [31] I. Ullah, Q.H. Mahmoud, Design and development of RNN anomaly detection model for IoT networks, *IEEE Access* 10 (2022) 62722–62750. <https://ieeexplore.ieee.org/document/9777970/>. <https://doi.org/10.1109/ACCESS.2022.3176317>
- [32] K. Saurabh, S. Sood, P.A. Kumar, U. Singh, R. Vyas, O.P. Vyas, R. Khondoker, LBDMIDS: LSTM Based Deep Learning Model for Intrusion Detection Systems for IoT Networks, 2022, [arXiv: 2207.00424 \[cs\]](https://arxiv.org/abs/2207.00424).
- [33] M.H. Jamal, M.A. Khan, S. Ullah, M.S. Alshehri, S. Almakdi, U. Rashid, A. Alazeb, J. Ahmad, Multi-step attack detection in industrial networks using a hybrid deep learning architecture, *Math. Biosci. Eng.* 20 (8) (2023) 13824–13848. <https://www.aimspress.com/article/doi/10.3934/mbe.2023615>. <https://doi.org/10.3934/mbe.2023615>
- [34] N. Dalvi, P. Domingos, S. Mausam Sanghai, D. Verma, et al., Adversarial classification, in: *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, Seattle WA USA, 2004, pp. 99–108. <https://dl.acm.org/doi/10.1145/1014052.1014066>. <https://doi.org/10.1145/1014052.1014066>
- [35] S.-M. Moosavi-Dezfooli, A. Fawzi, P. Frossard, et al., DeepFool: a simple and accurate method to fool deep neural networks, 2016, [arXiv:1511.04599 \[cs\]](https://arxiv.org/abs/1511.04599).
- [36] S. Calzavara, C. Lucchese, G. Tolomei, Adversarial training of gradient-boosted decision trees, in: *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, ACM, Beijing China, 2019, pp. 2429–2432. <https://dl.acm.org/doi/10.1145/3357384.3358149>. <https://doi.org/10.1145/3357384.3358149>
- [37] M.R. Naeem, T. Lin, H. Naeem, F. Ullah, S. Saeed, Scalable mutation testing using predictive analysis of deep learning model, *IEEE Access* 7 (2019) 158264–158283. <https://ieeexplore.ieee.org/document/8886413/>. <https://doi.org/10.1109/ACCESS.2019.2950171>
- [38] M.G. Merayo, M. Núñez, I. Rodríguez, Formal testing from timed finite state machines, *Comput. Netw.* 52 (2) (2008) 432–460. <https://www.sciencedirect.com/science/article/pii/S1389128607002812>. <https://doi.org/10.1016/j.comnet.2007.10.002>
- [39] M.H. Hassoun, *Fundamentals of Artificial Neural Networks*, MIT Press, Cambridge, Mass, Cambridge, Mass, 1995.
- [40] D.E. Rumelhart, G.E. Hinton, R.J. Williams, Learning representations by back-propagating errors, *Nature* 323 (6088) (1986) 533–536. <https://www.nature.com/articles/323533a0>. <https://doi.org/10.1038/323533a0>
- [41] S. Hochreiter, Long short-term memory, *Neural Comput.* MIT-Press (1997).
- [42] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, et al., Attention Is All You Need, 2023, [arXiv:1706.03762 \[cs\]](https://arxiv.org/abs/1706.03762).
- [43] Espressif Systems, ESP32 Series Datasheet, 2023. Accessed: 2025-02-14, https://documentation.espressif.com/esp32_datasheet_en.pdf.
- [44] P.P. Ioulianos, V.G. Vassilakis, M.D. Logothetis, et al., Battery drain denial-of-service attacks and defenses in the Internet of Things, *J. Telecommun. Inf. Technol.* 2 (2019) 37–45. <https://jtit.pl/jtit/article/view/554>. <https://doi.org/10.26636/jtit.2019.131919>
- [45] Y. Feng, W. Wang, Y. Weng, H. Zhang, A replay-attack resistant authentication scheme for the Internet of Things, in: 22017 IEEE International Conference on Computational Science and Engineering (CSE) and IEEE International Conference on Embedded and Ubiquitous Computing (EUC), IEEE, Guangzhou, China, 2017, pp. 541–547. <http://ieeexplore.ieee.org/document/8005853/>. <https://doi.org/10.1109/CSE-EUC.2017.101>
- [46] H. Wong, T. Luo, Man-in-the-middle attacks on mqtt-based iot using bert based adversarial message generation, in: *KDD 2020 AIoT Workshop*, 8, 2020.
- [47] A.H. Celdrán, P.M.S. Sánchez, C. Feng, G. Bovet, G.M. Pérez, B. Stiller, Privacy-preserving and syscall-based intrusion detection system for IoT spectrum sensors affected by data falsification attacks, *IEEE Internet of Things Journal* 10 (10) (2022) 8408–8415. Publisher: IEEE.
- [48] A.B. Lopez, K. Vatanparvar, A.P. Deb Nath, S. Yang, S. Bhunia, M.A. Al Faruque, A security perspective on battery systems of the Internet of Things, *J. Hardw. Syst. Secur.* 1 (2) (2017) 188–199. <http://link.springer.com/10.1007/s41635-017-0007-0>. <https://doi.org/10.1007/s41635-017-0007-0>
- [49] S. Shapsough, F. Aloui, I.A. Zualkarnan, Securing low-resource edge devices for IoT systems, in: 2018 International Symposium in Sensing and Instrumentation in IoT Era (ISSI), IEEE, 2018, pp. 1–4. Place: Shanghai, <https://ieeexplore.ieee.org/document/8538135/>. <https://doi.org/10.1109/ISSI.2018.8538135>
- [50] A.P. Haripriya, K. Kulothungan, Secure-MQTT: an efficient fuzzy logic-based approach to detect DoS attack in MQTT protocol for internet of things, *EURASIP J. Wirel. Commun. Netw.* 2019 (1) (2019) 90. <https://jwcn-eurasipjournals.springeropen.com/articles/10.1186/s13638-019-1402-8>. <https://doi.org/10.1186/s13638-019-1402-8>
- [51] I. Vaccari, M. Aiello, E. Cambiaso, SlowITe, a novel denial of service attack affecting MQTT, *Sensors* 20 (10) (2020) 2932. <https://www.mdpi.com/1424-8220/20/10/2932>. <https://doi.org/10.3390/s20102932>
- [52] F. Hussain, S.G. Abbas, U.U. Fayyaz, G.A. Shah, A. Toqeer, A. Ali, Towards a universal features set for IoT botnet attacks detection, in: 2020 IEEE 23rd International Multitopic Conference (INMIC), 2020, pp. 1–6. <https://doi.org/10.1109/INMIC50486.2020.9318106>
- [53] P.E. McKnight, J. Najab, Kruskal-Wallis test, in: I.B. Weiner, W.E. Craighead (Eds.), *The Corsini Encyclopedia of Psychology*, Wiley, 1 edition, 2010, pp. 1–1. <https://onlinelibrary.wiley.com/doi/10.1002/9780470479216.corpsy0491>. <https://doi.org/10.1002/9780470479216.corpsy0491>
- [54] H.H. Alshammari, The internet of things healthcare monitoring system based on MQTT protocol, *Alex. Eng. J.* 69 (2023) 275–287. <https://www.sciencedirect.com/science/article/pii/S1110016823000881>. <https://doi.org/10.1016/j.aej.2023.01.065>

- [55] C.-Y. Chen, S.-H. Wu, B.-W. Huang, C.-H. Huang, C.-F. Yang, Web-based Internet of Things on environmental and lighting control and monitoring system using node-RED, MQTT and Modbus communications within embedded Linux platform, *Internet of Things* 27 (2024) 101305. <https://www.sciencedirect.com/science/article/pii/S2542660524002464>. <https://doi.org/10.1016/j.iot.2024.101305>
- [56] T. Dao, M. Nguyen, S. Do, H. Tran, Cyberscurity Threats and Defense Mechanisms in IoT network, 2026, [arXiv:2601.00556](https://arxiv.org/abs/2601.00556).